

Отчёт к ЛР №4. ИГИ. Работа с файлами, классами, сериализаторами, регулярными выражениями и стандартными библиотеками
Луканский Дмитрий, гр.453502
Вариант 14
Задание 1.

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: forest_task
Implements the menu-driven user interface for the Forest monitoring program.
Inherits from common.Task and uses common.Action and common.ui helpers.
'''

from common.task import Task
from common.action import Action
from common.ui import read_int, read_str, show_table
from .forest import Forest
from .tree_species import TreeSpecies

class Task1(Task):
    """
    Main task for the Forest lab.
    Provides menu options: add species, show species info, summary, CSV/Pickle save/load.
    """

    name = "Task 1: Forest Monitoring"

    def __init__(self):
        """Initialize an empty forest."""
        self.forest = Forest()
```

Задание 2.

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: task2
Implements Task2 for text analysis.
Reads text from a file, analyzes using regular expressions,
displays results, saves to a file, archives with zipfile.
'''

import os
import zipfile
from datetime import datetime
from common.task import Task
from common.action import Action
from common.ui import read_str
from .text_analyzer import TextAnalysisMixin

class Task2(Task, TextAnalysisMixin):
    """Text analysis task implementing all required features."""
    name = "Task 2: Text Analysis"

    def __init__(self):
        self.input_filename = None
        self.text = ""
        self._analysis_results = {}

    @property
    def _text(self) -> str:
```

Задание 3.

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: statistics_mixin
Provides a mixin class for statistical calculations on a list of numbers.
'''

from typing import List, Union
from collections import Counter
import math

class StatisticsMixin:
    """
    Mixin that adds statistical methods to a class.
    Requires self._data_list (list of numeric values) to be present.
    """

    def mean(self) -> float:
        """Arithmetic mean of the data."""
        if not self._data_list:
            return 0.0
        return sum(self._data_list) / len(self._data_list)

    def median(self) -> float:
        """Median of the data."""
        n = len(self._data_list)
        if n == 0:
            return 0.0
```

Задание 4.

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: shape
Defines abstract base class Shape and its descendants: Rectangle and Square.
'''

from abc import ABC, abstractmethod
import math
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from .color import Color

class Shape(ABC):
    """
    Abstract base class for all geometric figures.
    Requires implementation of area() method.
    """

    @abstractmethod
    def area(self) -> float:
        """
        Calculate the area of the shape.
        Must be overridden in child classes.
        """
        pass
```

Задание 5.

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: task5
Demonstrates NumPy capabilities: array creation, indexing, slicing,
universal functions, and statistical operations (mean, median, corrcoef,
var, std). Also computes sum below main diagonal and std of diagonal
elements (two ways).
'''

import numpy as np
from common.task import Task
from common.action import Action
from common.ui import read_int

class Task5(Task):
    name = "Task 5: NumPy Matrix Analysis"

    def __init__(self):
        self.matrix = None
        self.n = 0
        self.m = 0

    def menu_text(self) -> str:
        if self.matrix is not None:
            return f"Matrix shape: {self.n} x {self.m}\nValues: 0..100"
```

Задание 6.

a)

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: task6b
Pandas analysis of Melbourne Housing dataset.
Task: Compare average price for max rooms vs min rooms.
'''

'''
Module: task6a
Demonstrates Pandas Series, DataFrame, and MultiIndex operations
using the Melbourne Housing dataset.
'''

import pandas as pd
from common.task import Task
from common.action import Action

class Task6a(Task):
    name = "Task 6a: Pandas MultiIndex Series (Melbourne Housing)"

    def __init__(self):
        self.df = None
        self.price_series = None

    def menu_text(self) -> str:
        if self.df is not None:
```

б)

```
'''
Laboratory work number: 4.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.04.2026.

Module: task6b
Pandas analysis of Melbourne Housing dataset.
Task: Compare average price for max rooms vs min rooms.
'''

import pandas as pd
from common.task import Task
from common.action import Action

class Task6b(Task):
    name = "Task 6b: Melbourne Housing Analysis"

    def __init__(self):
        self.df = None

    def menu_text(self) -> str:
        if self.df is not None:
            return f"Dataset loaded: {self.df.shape[0]} rows, {self.df.shape[1]} columns"
        else:
            return "No dataset. Please load Melbourne Housing CSV."

    def actions(self):
        return super().actions() + [
            Action("Load Melbourne Housing CSV", self.load_data),
```